FORGE FDE ACADEMY · SPRINT 2 · THE FDE MINDSET

Anomaly detection, the FDE way

This is not about how to build a fraud model. It is about how you make one operationally useful, explainable, measurable, safe and defensible — which is the part a client is actually paying you for.

One hour | 20 ideas | 1 case to solve | 7 rules to keep

Techademy · FORGE FDE Academy | Trainer: Lakshminarayanan

THE ONE SENTENCE TO LEAVE WITH

An anomaly score does not make a decision.

It decides which human should look at the case, how soon, and why.

If you can say this and defend it at the end of the hour, you have got what matters — even if you remember nothing else about the tooling.

01

Starting with the story, not with the ML

Sit with this before you think about any model

An insurer processes 4,000 motor claims a month. Finance finds 38 claims paid to only 4 bank accounts. Same garages. Similar amounts, all just below the review threshold.

And every individual claim was legitimate.

Now ask yourself: would the Sprint 1 document-reading system you built have caught this? The answer is no.

SPRINT 1 asks

```
One claim
   ↓
Read documents
   ↓
Extract facts
   ↓
Build profile

"Is THIS claim valid?"
```

SPRINT 2 asks

```
Many claims
   ↓
Look for relationships
   ↓
Detect unusual patterns
   ↓
Score risk
   ↓
Put it in the right human queue
```

THIS IS THE MOST IMPORTANT SHIFT IN THE WHOLE SPRINT

Everything else is machinery. If you miss this, the machinery has nowhere to go.

02

The FDE's job is not to invent fraud rules

What a junior engineer reaches for

"Let's build an ML model and see what it finds."

A reasonable instinct, and the wrong starting point. It skips the only people who already know the answer.

What you should say instead

"First, let's talk to the people who understand the business."

Ask a senior claims handler what makes them suspicious. They will tell you in five minutes. Nobody has ever written it down.

The translation you are being paid to do

HUMAN KNOWLEDGE

"I get suspicious when a claim comes very late and it is almost at the policy limit."

→

MACHINE RULE

```
IF  days_since_incident   > 21
AND estimate_vs_limit_pct > 90%
THEN add 30 risk points
```

THE THING TO BE CLEAR ABOUT

This is requirements engineering, not machine learning. You are not inventing risk logic — you are extracting it from people who already have it, and encoding it so it runs on all 4,000 cases instead of only the ones a good handler happens to see.

03

Four words, and what they actually mean

Event = what happened · Entity = who it happened to · Variable = facts about it · Detector = the thing giving an opinion

The insurance example

```
EVENT       A claim arrived

ENTITY      Claimant / policy /
            garage / bank account

VARIABLES   Claim amount = ₹4.8L
            Days since incident = 26

DETECTOR    Risk detector
```

The same shape, in a hospital

```
EVENT       Patient arrives

ENTITY      Patient / doctor /
            hospital

VARIABLES   Age
            Temperature
            Blood pressure

DETECTOR    Triage assessment
```

WHY THE HOSPITAL VERSION IS WORTH A MOMENT

The architecture is not about fraud, or insurance, or Azure. And it carries a lesson you will need on every engagement: words like event type, entity and detector belong to one managed product, not to Azure ML. Do not get trapped in a vendor's vocabulary — the method is portable, the button names change every year.

04

The eight variables are really eight questions

These are not a list to memorise. Each one is a question you are asking about a claim.

Variable	The question we are actually asking	Describes
declared_income	Does the financial profile make sense?	the claim
estimate_vs_limit_pct	Is it suspiciously close to the limit?	the claim
days_since_incident	Reported unusually late, or early?	the claim
policy_age_days	A large claim soon after the policy started?	the claim
application_velocity_30d	Is this person generating too many claims?	RELATIONSHIP
device_fingerprint_reuse	Same device used for many people?	RELATIONSHIP
bank_account_reuse_count	Money going to the same account repeatedly?	RELATIONSHIP

THE LAST COLUMN IS THE INSIGHT, NOT THE LIST

Four variables describe the claim. Four describe its RELATIONSHIP to other claims. Anomaly detection becomes powerful in the second group — and those are exactly the ones that would have caught Meridian.

05

Two tests for every variable — first: can you know it at scoring time?

Here is the trap, in a form you will recognise immediately.

the trap

```
You build a model to predict
which students will fail.

You give it:
   Attendance
   Previous marks
   Assignments
   FINAL EXAM MARK      ←

Accuracy: 99%
```

Is 99% good?
It looks excellent. Now ask when the final exam mark actually becomes available.

The model cheated
It used something that does not exist at the moment you need the prediction.

Now back to claims
“Final settlement amount” is known only after settlement. It cannot score a claim that has just arrived.

This has a name
Leakage. It is the most common reason a model looks brilliant in testing and fails in production.

NOW APPLY IT TO YOUR OWN EIGHT

Three of your eight variables need a live lookup. Work out which three — then ask what happens if that lookup runs as an overnight batch. The scoring is a day late and the variable is useless.

A variable you only know AFTER the outcome cannot help you predict the outcome. Obvious when stated. Very easy to miss in a feature list of forty columns.

06

Second test — is the variable fair to use?

The obvious part is easy: do not use age, gender or ethnicity. The part that will catch you out is more subtle.

A variable does not have to LOOK discriminatory to become a proxy.

The variable

garage_claim_share_90d = 35%

Looks completely harmless. It is a business fact about claim concentration.

The context

In a small town, Garage A serves one particular community heavily, and that community has a geographic concentration.

The consequence

The model may learn something you never intended, and it will be very hard to detect afterwards. It does not know the variable is sensitive.

ASK THE BETTER QUESTION

Not “is this variable technically allowed?” — that has an easy yes. Ask instead: “What does this variable represent in THIS geography and THIS business?”

The first question is answered by a policy document. The second needs somebody who understands the client's market — and it is the one that keeps you out of trouble.

07

Why rules come before ML

NOT THIS

`Data → ML → magic`

THIS

```
Business knowledge
      ↓
Explicit rules
      ↓
Understand the problem
      ↓
Training labels
      ↓
ML model
      ↓
Rules + model
```

Rules = things we already know

"Claims above ₹5 lakh must always be reviewed." That is a business OBLIGATION. You do not want a model deciding that today ₹5 lakh is not risky.

Model = things we do not yet know

New policy + same garage + near-limit estimate + high frequency is an unusual COMBINATION. Nobody described it. That is where ML earns its place.

THE ARGUMENT THAT WINS THIS DISCUSSION WITH A CLIENT

If the client cannot explain what "suspicious" means, how are they going to produce useful training labels? Rules-first is not conservatism. It is your feasibility test.

Rules encode obligations. Models encode patterns.

08

The scoring example — and the mistake to catch in yourself

claim #103

```
Late + near limit          +30
Repeated bank account      +45
New policy + high value    +35
                           ---
                           110
```

The instinct is to say:
"Risk = 110, therefore reject."

STOP THERE.
That is the exact mistake this whole sprint exists to prevent.

what should happen instead

```
Risk score = 110   →   HIGH RISK   →   PRIORITY REVIEW   →   HUMAN INVESTIGATES
```

Score ≠ decision

OR, MORE BLUNTLY "The model does not fire the employee. It fires the queue."

09

AUC is not the business outcome

Picture this conversation, because you will have it.

Your model has AUC = 0.91. Everyone celebrates.
Then the Operations Manager asks: “How many claims do I have to review tomorrow?”
And the engineer says: “Uh… 0.91?”

what the client actually cares about

```
How many cases get flagged?
          ↓
How many are actually suspicious?
          ↓
How many false alarms?
          ↓
How many reviews per day?
          ↓
Can my team handle them?
```

```
4,000 claims / month
≈ 133 claims / day

Threshold produces 16 reviews / day
```

Operations needs to know:
“Can my two reviewers handle 16 extra cases every day?”

That is a capacity question, not an ML question.

THE RULE TO WORK BY

Never take an AUC to an operations meeting. Take precision at your chosen threshold and the review volume per day. Those two numbers decide whether the system gets used or quietly switched off.

10

Think about precision as a queue, not a formula

Forget the formula. Picture the queue.

you flag 100 claims. after investigation:

```
25  =  genuinely suspicious
75  =  perfectly fine

Precision = 25%
```

What that means to the person doing the work:

"For every 100 cases you send me, 75 are a waste of my morning."

Precision asks: "Of what I flagged, how much was real?"

Recall asks: "Of what was real, how much did I catch?"

Precision is what your OPERATIONS team feels

Of the cases you flagged, how many were real. Get this wrong and the queue gets ignored within a fortnight.

Get precision wrong → **reviewers drown in false alarms.**

Recall is what your RISK team worries about

Of all the fraud out there, how much did you catch. Get this wrong and losses continue quietly.

Recall = **How much of the real risk did we catch?**

AND THEY PULL IN OPPOSITE DIRECTIONS

Raise the threshold and precision improves while recall falls. Lower it and the reverse. There is no setting that fixes both, which is exactly why the threshold is a business decision rather than an engineering one.

The FDE's job is to balance both against business capacity and the cost of missed risk.

11

Selection bias — the ceiling on what you can promise

Your training data has 1,850 legitimate cases and 150 fraud. Ask yourself where those 150 came from.

the police analogy

```
Police record only the thieves
they CATCH.

You train an AI on those records.

The AI becomes excellent at spotting
people who look like the thieves
police previously caught.

What about the ones never caught?
```

They came from fraud somebody DETECTED
Not from all fraud. From the subset that was found.

So the model learns a narrower thing
“What does previously discovered fraud look like?” — not “what does fraud look like?”

More data of the same kind will not fix it
This is the part people resist. It is a property of how your labels were made, not of the sample size.

So it gets documented, not hidden
It belongs in the model card, in plain words, next to the metrics.

THIS IS THE MOST SUBTLE IDEA IN THE SPRINT

It is also the one your client most needs to hear, because it sets the ceiling on what any fraud model can promise. If you have never said it out loud, you will be asked about it in a model risk review and you will not have an answer ready.

You can only learn from the fraud somebody already found.

12

Day 28 — where your engineering meets governance

OLD SYLLABUS

```
Score = 82
      ↓
HIGH  →  REJECT
```

The governance decision from Sprint 0 says a person must make the decision. “Reject” is a decision. So is “auto-approve”.

WHAT IT SHOULD DO

```
Score = 82
      ↓
HIGH
      ↓
PRIORITY REVIEW
      ↓
Reasons attached
      ↓
Linked cases shown
      ↓
HUMAN DECIDES
```

`approve / reject  →  straight_through / standard_review / priority_review`

THIS IS NOT JUST WORDING

Your system is SUPPORTING a decision rather than MAKING one. That distinction keeps an automated risk score out of scope for automated-decision-making obligations — and the code is identical either way. Only the names and the downstream behaviour change.

13

The three queues you are building

`SCORE = 22`

↓

STRAIGHT THROUGH

- Processed without extra scrutiny
- A human still signs off
- This is the volume path

`SCORE = 51`

↓

STANDARD REVIEW

- Goes into the normal queue
- Reasons attached
- Handled in turn

`SCORE = 83`

↓

PRIORITY REVIEW

- Top of the queue
- All triggered rules listed
- Linked cases shown alongside

30 AND 70 ARE PLACEHOLDERS, AND YOU MUST SAY SO

The real thresholds come from detection performance, false alarm volume, review capacity and operational risk. That is a business decision, not an engineering one — and if you pick the numbers yourself you have quietly taken a decision that belongs to the client.

A question worth sitting with

If the client can only staff four reviews a day, where do you put the thresholds — and what does that do to your detection rate? Trading those two against each other is the moment this whole sprint becomes concrete.

14

Never fail open

What happens if the scoring service is down? Here are the two lines you would probably have written.

THE DANGEROUS DEFAULT

```
try:    score = call_scoring(case)
except: score = 0
```

Every case now looks low risk. Nothing alerts.

WHAT IT SHOULD DO

```
Scoring unavailable
        ↓
Route to STANDARD REVIEW
        ↓
Mark the case  scored = false
```

Fail towards the human, never away from them

If you cannot score a case, a person looks at it. That costs reviewer time during an outage, which is the correct trade.

Record that it was NOT scored

Otherwise somebody later assumes the case was checked and cleared. An unchecked case that looks checked is worse than one that looks unchecked.

WHY THIS DESERVES FIVE MINUTES OF YOUR ATTENTION

Two lines of code, written by accident, silently disabling your entire control during exactly the period when things are already going wrong — and no test catches it unless you deliberately wrote one. Have you written that test?

15

Reasons matter more than the score

USELESS TO A HANDLER

```
Risk score: 72
```

Where do they start?

ACTIONABLE

```
Risk score: 72

• Same bank account: 12 claims
• Same garage: 31% of local claims
• Claim filed 26 days after incident
```

The handler knows where to look

"Same bank account, 12 claims" tells them to pull the other eleven before anything else.

The client can audit the decision

A reason is reviewable. A number is not.

The regulator can be answered

"Flagged because of repeated bank account reuse" is a sentence that survives scrutiny.

Explainability is not a nice-to-have. It is the product.

A QUESTION TO TEST YOUR OWN DESIGN AGAINST

Your queue has forty cases in it this morning, and each one shows only a number. How long before the reviewer stops trusting it and just works through them in date order? Usually about a week — and at that point you have built a very expensive sorting mechanism nobody uses.

16

Meridian, end to end

Sprint 1 only	**No**	Each claim was individually valid. Nothing to see.
Day 26 rules	**Yes, eventually**	repeated_bank_account fires at the 3rd claim. Claims 1 and 2 still went through.
Day 27 model	**Earlier**	New-ish policy + same garage + near-limit estimate scores high on claim 2.
Day 28 routing	**And acted on**	Priority review, 2 linked claims shown alongside. A handler sees it in 30 seconds.

BE HONEST ABOUT WHAT "CAUGHT" MEANS, OR YOUR CLIENT WILL HEAR A PROMISE YOU DID NOT MAKE

Even in the best case, claims 1 and 2 were paid before anything fired. An anomaly system reduces losses; it does not prevent the first instance of a new pattern.

Say that at the START of an engagement, not after the first incident. It is the difference between a realistic expectation and a disappointed client.

17

The five-layer mental model

If you can draw this from memory at the end of the sprint, you have understood it.

1	**BUSINESS UNDERSTANDING**	*what does suspicious mean here?*
2	**VARIABLES**	*what facts can we observe, and may we use them?*
3	**RULES**	*what do we already know?*
4	**MODEL**	*what do we not yet know?*
5	**ROUTING**	*what should a human do about it?*

NOTE THE DIRECTION OF TRAVEL

It starts with the business and ends with a human. The model is layer four of five — important, and not the point.

18

Ten questions you should be able to answer

1. Why not just use ML from the start?
2. Why do rules still matter after we have a model?
3. Why is 0.95 AUC not automatically good news?
4. Why is a high-recall model sometimes operationally useless?
5. What happens if the client cannot staff the review queue?
6. Why must the system never auto-reject?
7. What is the risk of using garage share as a variable?
8. How do you explain a false positive to a customer?
9. What if the model degrades after six months?
10. Who signs off the thresholds — and on what evidence?

USE THESE ON YOURSELF, AND ON YOUR CLIENT

If you can answer all ten without hesitating, you are ready to run this conversation on an engagement. Question 5 is the one that changes how most people think about the whole sprint.

19

Seven rules to take away

1	**Anomaly detection supports humans**	It never replaces them.
2	**Rules encode obligations**	Models encode patterns.
3	**Explainability is mandatory**	Not optional, and not a later phase.
4	**Metrics must map to workload**	An AUC is not a staffing plan.
5	**Bias risk is architectural**	Not accidental, and not the model's fault.
6	**Fail safe, not silent**	Route to a human, and record that you did.
7	**The score selects a queue**	It does not select an outcome.

IF YOU REMEMBER ONLY ONE

Rule 7 contains the other six.

20

How the next hour goes

5 min	**The Meridian story**	No slides. Just the problem, and a question.
5 min	**What would Sprint 1 have caught?**	Work it out before you are told.
10 min	**Event / entity / variable / detector**	Then the same shape in a hospital.
10 min	**You build 3 rules, live**	From your own experience, not from the slide.
10 min	**Leakage and fairness**	The exam trap, then the garage problem.
10 min	**Precision, recall and workload**	The queue of 100, never the formula.
5 min	**The three-queue routing**	And the word you are going to change.
5 min	**The crux**	“A score selects a queue, not an outcome.”

THE TWO HIGHLIGHTED ROWS ARE THE ONES THAT MATTER MOST

Building rules yourself is where this stops being a lecture. The crux is where it all lands.

★ Your turn — five minutes, no laptops

What should happen to this claim?

the claim

```
Claim amount          ₹4,80,000
Policy limit          ₹5,00,000
Days since incident    26
Policy age              18 days
Bank account reuse       7
Garage share            31%
```

Near limit	96% of the policy limit
Late report	26 days after the incident
Very new policy	18 days old
Repeated account	7 prior claims to it
Garage concentration	31% of local claims

If your first instinct was “reject it”, that instinct is the whole reason for this session.

THE CORRECT ANSWER, AND WHY IT MATTERS

PRIORITY REVIEW — not rejection. High risk, clear reasons, sent to a human quickly with the seven linked claims attached. This claim may still be perfectly genuine, and a person is the only one entitled to decide that.

THE FDE MINDSET · CLOSE

You are not building a fraud detector.

You are making a model operationally useful and safe to deploy.

01 Start with the story. The ML comes fourth of five.

02 Your job is to extract risk logic, not to invent it.

03 Every variable gets two tests: leakage and proxy.

04 Never take an AUC to an operations meeting.

05 A score selects a queue. It does not select an outcome.

If you can defend the crux, you can run this conversation on an engagement.

TAKE AWAY

Score ≠ decision
Two variable tests
Rules before ML
Reviews per day
Fail to the human